

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence **COURSE CODE:** CSA1707

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 50%

QUESTIONS: 5

Duration: 1 Hour
Total Marks: 50

Annexure A

Scenario Based Assignment

Code of Conduct:

I J O Joanna Divine (192572003) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: J O Joanna Divine

Answer 1: AI-Powered Emergency Medicine Delivery Drones

Scenario summary: A drone must deliver emergency medicines through a flood-affected, dynamic environment. It has a straight-line-distance heuristic, partial real-time information about blocked roads, and variable terrain/weather costs, while needing fast, low-risk routing decisions.

i. Behaviour of Greedy Best-First Search (GBFS)

GBFS expands the node that appears closest to the goal according to the heuristic $h(n)$ alone, ignoring the actual path cost $g(n)$ already incurred. In this scenario it would repeatedly pick the next waypoint with the smallest straight-line distance to the destination, which makes it fast and memory-light — useful when the drone must decide quickly.

- **Strengths:** Very fast decisions (important under urgency); low computational and memory overhead; works reasonably well when the heuristic is a good proxy for real difficulty.
- **Weaknesses:** Not optimal and not complete in general — it can be lured toward a point that looks close on a straight line but is actually blocked by floodwater or unsafe terrain, forcing long detours or dead ends. Because it ignores accumulated cost, it can select a route with lower straight-line distance but far higher real travel risk or fuel cost, which is dangerous when uncertainty is high.

ii. Behaviour of A* Search

A* evaluates nodes using $f(n) = g(n) + h(n)$, combining the real cost incurred so far (g) with the estimated remaining cost (h). This lets the drone balance 'how much effort has this route already cost' against 'how promising does the remaining path look'. As long as the straight-line-distance heuristic is admissible (never overestimates the true remaining cost), A* is guaranteed to find the optimal path, correctly weighing safer-but-slightly-longer routes against risky shortcuts.

iii. Impact of Heuristic Accuracy

- **Greedy Best-First Search:** Heavily dependent on heuristic accuracy since $h(n)$ is the only signal used. A poor heuristic (e.g., one that ignores terrain/weather) can send the drone toward badly costed or blocked routes with no correction mechanism.
- **A*:** More robust to heuristic imperfections because $g(n)$ still anchors the search in actual cost. A more accurate (but still admissible) heuristic reduces the number of nodes A* must expand, making it faster while preserving optimality. An inadmissible heuristic, however, can make A* sub-optimal or even unsafe, so heuristic design matters for both algorithms but is more forgiving for A*.

iv. Effect of Dynamic Changes (Blocked Paths)

Both algorithms are normally defined over a static graph, so a newly blocked road invalidates part of the previously computed path. In practice this requires re-planning: recomputing the search (or using incremental/real-time variants such as D* Lite) whenever new sensor updates arrive. GBFS re-plans quickly but may repeat the same heuristic-driven mistakes; A* re-planning is more computation-heavy per cycle but produces a locally optimal route given the latest known map, which matters more when the cost of a wrong detour (fuel, time, risk to the mission) is high.

v. Recommendation

A* (or a real-time incremental variant of it, such as D* Lite/Anytime A*) is the more suitable choice. Although GBFS is faster per decision, its disregard for actual path cost makes it unsafe and unreliable

under the terrain/weather variability and blocked-path uncertainty described here. A^* balances optimality with reasonable speed, and its cost-awareness directly protects against the two things that matter most in this scenario — delay and risk. For extremely tight time budgets, a bounded or weighted A^* variant can trade a small amount of optimality for faster response, while still outperforming pure greedy search on safety.

Answer 2: Smart City Traffic Signal Optimization

Formulation as an optimization problem: Each intersection's signal timing (green/red duration, phase offsets) is a decision variable. The state is the full vector of timings across all intersections, and the objective function combines total waiting time, fuel consumption, and congestion into a single cost (or weighted multi-objective cost) to be minimized. The search space is the set of all possible timing configurations across hundreds of intersections — combinatorially huge, continuous in nature, and constantly shifting as traffic patterns change. Traditional systematic search (BFS/DFS/A*) is inefficient here because there is no natural notion of a 'goal state' to search toward, the branching factor is enormous, and the environment changes faster than an exhaustive search could explore it — this calls for local/metaheuristic search that iteratively improves a candidate solution instead.

i. Hill Climbing in this Scenario

Hill Climbing starts from an initial signal-timing configuration and repeatedly moves to a neighboring configuration with a better (lower-cost) evaluation, stopping when no neighbor improves further. It is simple and cheap to run per intersection, which fits the need for continuous, lightweight adjustment. However it is a purely local, greedy method: once no local move improves the objective, it stops, even if a better global configuration exists elsewhere.

ii. Local Maxima, Plateaus, and Ridges

- **Local maxima/minima:** The signal timing may reach a configuration that is better than any single small tweak but is far from the true city-wide optimum — e.g., locally reducing wait at one junction while leaving nearby junctions congested.
- **Plateaus:** Several neighboring configurations may produce almost identical waiting times (e.g., shifting a green phase by a second or two changes nothing measurable), so Hill Climbing has no gradient to follow and can wander or stall.
- **Ridges:** Improving traffic flow may require simultaneous, coordinated changes at several intersections (e.g., a 'green wave' along a corridor); changing one signal at a time can make things temporarily worse before they get better, and simple Hill Climbing cannot see across such a ridge.

iii. Overcoming the Limitations

- **Simulated Annealing:** Occasionally accepts a worse configuration (with a probability that decreases over time/temperature) to escape local optima and cross plateaus, gradually converging toward better citywide solutions.
- **Genetic Algorithms:** Maintain a population of candidate timing plans, combine (crossover) and mutate them across generations, and select the fittest — this explores multiple regions of the search space in parallel, making it well suited to coordinating many intersections simultaneously and avoiding premature convergence to a single local optimum.

iv. Recommendation

A Genetic Algorithm (optionally combined with Simulated Annealing for local refinement, i.e., a hybrid 'memetic' approach) is the best fit. Traffic optimization here is large-scale, multi-objective, and requires coordinated changes across many intersections — properties that reward the parallel, population-based exploration of GAs. Simulated Annealing is a good lighter-weight alternative when compute resources are limited or when re-optimizing a single congested corridor in near real time, since it is simpler and needs less memory than maintaining a population. Both scale and adapt far

better than plain Hill Climbing to the dynamic, high-dimensional nature of city-wide traffic control.

Answer 3: Autonomous Mars Rover Navigation

i. Why an Online Search Agent is Required

Offline search assumes the agent has complete knowledge of the environment before it starts acting, so it can compute a full solution path in advance and simply execute it. The Mars rover does not have this luxury: it has no complete map of the terrain, limited sensing range, and must discover hazards (craters, rocks) as it moves. This is precisely the definition of an **online search** problem — the agent must interleave computation (deciding the next move) with actual action and sensing, updating its plan as new information arrives, because committing to a full offline plan could lead it straight into an unseen hazard.

ii. Challenges of Partial Observability and Dynamic Environments

- The rover only perceives its immediate surroundings, so it cannot guarantee a chosen action is safe beyond its sensing radius — it may commit to a path that later reveals a dead end or hazard.
- Actions can be costly or irreversible (e.g., driving into a crater risks damage), so exploration must be cautious rather than purely exploratory as in classical search.
- The terrain can shift (dust, small rockslides), meaning previously mapped 'safe' areas may need re-verification, so the rover cannot fully trust old information.
- Limited communication delay with Earth means the rover often cannot wait for human confirmation and must make local, safe decisions autonomously.

iii. Building and Updating the Internal Model

The rover maintains an internal map (e.g., an occupancy grid or graph of visited/known cells) that starts mostly unknown. As it moves and senses, it marks cells as free, hazardous, or unexplored, incrementally 'filling in' the map. When it detects a hazard or dead end, it updates the model and backtracks or replans from its current position rather than restarting from scratch — a strategy similar to the **LRTA*** (Learning Real-Time A*) approach, where the agent updates heuristic cost estimates for cells it has visited so it avoids retracing costly mistakes.

iv. Online Search vs. Uninformed and Informed Offline Search

- **Uninformed search (BFS/DFS):** Assumes the whole graph is known in advance and explores blindly without any distance estimate — completely impractical here since the terrain is unknown and exhaustive blind exploration on Mars would be extremely costly and slow.
- **Informed offline search (A*):** Needs a full map and heuristic computed ahead of time to guarantee optimality — but the rover cannot compute this because the environment is only partially known.
- **Online search (e.g., online DFS, LRTA*):** Explicitly designed for agents that must act before fully knowing the environment; it interleaves sensing, planning, and acting, trading some path optimality for feasibility and safety in unknown terrain — exactly what the rover needs.

v. Justification of the Most Suitable Approach

An online, informed search strategy such as LRTA* is the most suitable. It combines the adaptability required for partial observability and a changing environment (updating its knowledge as it senses new terrain) with heuristic guidance (using estimated distance/cost to prioritize promising directions), giving a good balance between safety (avoiding hazards it discovers only locally) and efficiency (not

wandering blindly like uninformed search). This directly addresses the rover's core constraints: uncertainty, the need to adapt to new information, and the high cost of unsafe moves.

Answer 4: University Exam Timetabling as a CSP

Formulation as a Constraint Satisfaction Problem

A CSP is defined by a set of variables, their domains, and the constraints relating them. Exam timetabling maps naturally onto this framework because the goal is to assign a discrete resource (a time slot and hall) to each exam such that a set of hard rules is respected.

i. Variables, Domains, and Constraints

Variables:

- One variable per exam/course to be scheduled, e.g., Exam■, Exam■, ..., Exam■ (each variable's value will represent its assigned time slot and hall).

Domains:

- For each exam variable, the domain is the set of all valid (time slot, exam hall) pairs available during the exam period — e.g., {Day1-9AM-HallA, Day1-9AM-HallB, Day1-2PM-HallA, ...}.

Constraints:

- **Unary constraints:** An exam requiring special accommodation must be assigned only to a slot/hall equipped for it.
- **Binary constraints:** Two exams taken by any common student cannot share the same time slot (no overlap); a subject that must precede another must be assigned an earlier slot.
- **Hall-capacity / global constraints:** The number of exams assigned to the same time slot cannot exceed the number of available halls, and each hall's seating capacity must not be exceeded.
- **Resource constraints:** A faculty member invigilating cannot be assigned to two exams in the same slot; faculty availability windows restrict which slots their exams can use.

ii. Backtracking Search in this Scenario

Backtracking search assigns exams to (slot, hall) values one variable at a time, checking constraints incrementally: after each assignment it verifies no student-overlap, hall-capacity, precedence, or faculty constraint is violated. If a violation is found, the search backs up and tries a different value for the most recently assigned exam (or an earlier one), continuing until either a complete valid timetable is found or all possibilities are exhausted. This is systematic and complete, but its efficiency depends heavily on the order in which exams and slot/hall values are tried.

iii. How Constraint Propagation Improves Efficiency

Techniques such as **forward checking** remove, from the domains of not-yet-assigned exams, any (slot, hall) value that would immediately conflict with the exam just assigned (e.g., removing that same time slot from every exam sharing a student with it). This shrinks the search space early and lets the algorithm detect a dead end (an exam left with an empty domain) far sooner than plain backtracking would, avoiding wasted exploration deep in the search tree. Stronger propagation (e.g., arc consistency, AC-3) can prune even more aggressively by ensuring every pair of related exams has at least one consistent combination of values before search continues.

iv. Real-World Challenges and Best Strategy for Scalability

- As the number of courses, students, and constraints grows, the search space grows combinatorially, and pure backtracking can become very slow.
- Constraints of different importance (hard rules like no double-booking vs. soft preferences like minimizing back-to-back exams for students) need to be handled differently — hard constraints must never be violated, while soft ones can be optimized rather than strictly enforced.
- Special accommodations and faculty availability add extra layers of unary/global constraints that must be respected without excessively restricting the search.

Best strategy: Backtracking search combined with constraint propagation (forward checking / arc consistency) and good heuristics — specifically the **Minimum Remaining Values (MRV)** heuristic to choose the most constrained exam next, and the **Least Constraining Value** heuristic to try slot/hall options that leave the most flexibility for other exams. This combination scales far better than naive backtracking alone, because it prunes the search space early and focuses effort on the hardest-to-place exams first, which is essential as the number of courses and students grows.

Answer 5: Strategic Game AI (Minimax & Alpha-Beta Pruning)

i. How Minimax Models the Problem

Minimax models the real-time strategy game as a two-player, zero-sum game tree in which the AI (the maximizing player) and the human opponent (the minimizing player) alternate moves. Each leaf node (or a cut-off depth) is scored by an evaluation function reflecting how favorable that game state is for the AI. The algorithm works backward from these values: at MAX nodes it picks the move with the highest value (assuming the AI will play optimally), and at MIN nodes it picks the move with the lowest value (assuming the opponent will also play optimally and try to minimize the AI's outcome). This produces a move that is optimal against a worst-case, perfectly rational opponent.

ii. Computational Challenges of Large Game Trees

- The number of possible game states grows exponentially with search depth and branching factor (the number of legal moves at each turn), so exploring the full tree to the end of the game is computationally infeasible for a real-time strategy game with many units and move options.
- Strict real-time limits mean the AI cannot afford to fully explore even a modest depth before a decision is due, forcing a trade-off between search depth (look-ahead quality) and response time.
- Memory requirements to store or revisit large portions of the tree can become prohibitive without careful depth limiting and pruning.

iii. How Alpha-Beta Pruning Improves Efficiency

Alpha-Beta pruning augments Minimax with two bounds — alpha (the best value the maximizer can guarantee so far) and beta (the best value the minimizer can guarantee so far) — passed down the tree during search. Whenever, at a given node, the algorithm finds a value that is worse for one player than a value already guaranteed elsewhere in the tree ($\alpha \geq \beta$), it can safely stop exploring the remaining branches of that node, because a rational opponent would never allow the game to reach that branch. Crucially, this pruning does not change the final decision — it produces exactly the same result as full Minimax while skipping large parts of the tree that cannot affect the outcome. With good move ordering (evaluating the most promising moves first), Alpha-Beta pruning can reduce the effective branching factor roughly from b to about $\sqrt{b}$ in the best case, roughly doubling the depth that can be searched within the same time budget.

iv. Designing an Evaluation Function

Since the tree cannot be searched to completion in real time, a heuristic evaluation function scores non-terminal states at the search cut-off depth. Relevant factors for a real-time strategy game include:

- **Material/unit balance:** number and strength of the AI's units versus the opponent's.
- **Resource control:** amount of resources gathered, territory or key map objectives held.
- **Positional strength:** unit positioning relative to choke points, high ground, or defensive structures.
- **Economic/production capacity:** buildings, upgrade levels, and production rate, since these predict future strength beyond the current snapshot.
- **Threat/vulnerability assessment:** exposure of high-value units, incoming attacks, or unguarded bases.

These factors are typically combined as a weighted sum, with weights tuned (manually or via learning) so the function reliably distinguishes stronger from weaker positions without being too expensive to compute at every node, since it must be evaluated many times per move.

v. Strategy for Balancing Optimality and Real-Time Performance

- Use **depth-limited Minimax with Alpha-Beta pruning** rather than searching to the end of the game, choosing a depth that fits the time budget.
- Apply **iterative deepening**: search depth 1, then 2, then 3, and so on, always keeping the best move found so far, so the AI can be interrupted at any time and still return a reasonable decision.
- Order moves using cheap heuristics (e.g., try captures or high-impact moves first) to maximize the benefit of Alpha-Beta pruning and reach deeper search within the same time limit.
- Cache previously evaluated positions (transposition tables) to avoid recomputation when the same state is reached via different move orders.

This combination lets the AI make well-reasoned, near-optimal decisions while strictly respecting the real-time constraint, since the search can always be cut off gracefully without losing the best answer found up to that point.